

PART 1:

a) My Hack Assembler relies on three main modules to translate assembly code into machine code. Here is the API for each:

Parser Module Encapsulates access to the input code. It reads an assembly language command, parses it, provides convenient access to the command's components, and ignores all whitespace and comments.

- A. `__init__(file_path)`: Opens the input file and gets ready to parse it.
- B. `hasMoreLines()`: Returns a boolean indicating if there are more commands left in the input file.
- C. `advance()`: Reads the next command from the input and makes it the current command. Should be called only if `hasMoreCommands()` is true.
- D. `instruction_type()`: Returns values 0, 1 or 2 to indicate the type of the current command as `a_instruction`, `c_instruction`, or `l_instruction` respectively.
- E. `symbol()`: Returns the symbol or decimal value of the current command. It is called only for a or l instruction.
- F. `dest()`: Returns the destination mnemonic in the current `c_instruction`.
- G. `comp()`: Returns the computation mnemonic in the current `c_instruction`.
- H. `jump()`: Returns the jump mnemonic in the current `c_instruction`.

Code Module Translates Hack assembly language mnemonics into their corresponding binary codes.

- A. `dest(mnemonic)`: Returns the 3-bit binary code of the given dest mnemonic.
- B. `comp(mnemonic)`: Returns the 7-bit binary code of the given comp mnemonic.
- C. `jump(mnemonic)`: Returns the 3-bit binary code of the given jump mnemonic.

SymbolTable Module Maintains the correspondence between symbolic labels (variables and (LOOP) labels) and their numeric memory addresses.

- A. `__init__()`: Creates a new, empty symbol table.
- B. `addEntry(symbol, address)`: Adds the (symbol, address) pair to the table.
- C. `contains(symbol)`: Returns a boolean indicating whether the symbol table already contains the given symbol.
- D. `getAddress(symbol)`: Returns the numeric address associated with the given symbol.

b) According to the Hack machine language convention, custom variable symbols are mapped sequentially into memory starting at RAM address 16. The addresses 0 through 15 are reserved for the predefined virtual registers (R0 through R15).

PART 2:

2.

- a) To store matrices A, B, and C, the program requires a total of 27 memory registers. Since A, B, and C are 3×3 matrices, each matrix consists of 9 distinct elements. 9 registers per matrix multiplied by 3 matrices equals 27 registers total.
- b) The Hack ALU lacks a dedicated hardware multiplier and natively supports only basic operations like addition, subtraction, bitwise AND/OR, and negation. Therefore, we must implement multiplication via software in the assembly code. The two primary design choices are:
 - Repeated Addition: Computing $X * Y$ by adding X to a sum Y times.
 - Shift-and-Add: Mimicking binary multiplication by shifting bits left and conditionally adding, which completes in a fixed $O(16)$ time for 16-bit numbers.

For simplicity of implementation, code readability, and ease of debugging, I chose the repeated addition approach for this program, even though it is less computationally efficient than the bitwise method.

c)

Static Initialization: This occurs when the matrix values are hardcoded directly into the program at compile time. To do this, we would write a series of A-instructions and C-instructions directly inside the .asm file before the multiplication logic begins.

Dynamic Initialization: This occurs when the memory is populated at runtime or by an external agent, allowing the same assembly code to process different data without being rewritten.

PART 3:

3.

- a) Yes, the choice of storing matrices in row-major versus column-major order affects the efficiency of the assembly program, specifically regarding pointer arithmetic inside the multiplication loops.

In Hack assembly, sequential memory access is highly efficient, requiring only a simple increment ($M=M+1$) to advance a pointer to the next element. Conversely, jumping across a row to read down a column requires multiple instructions to calculate the offset and add it to the pointer, leading to loss of efficiency.

The standard matrix multiplication algorithm calculates the dot product of a row from Matrix A and a column from Matrix B.

- Matrix A is traversed sequentially across its rows. Therefore, storing Matrix A in row-major order is the most efficient choice, allowing the inner loop pointer to simply increment by 1.
- Matrix B is traversed sequentially down its columns. If stored in row-major order, the pointer must jump by the matrix width every iteration. By storing Matrix B in column-major order, the memory becomes sequentially aligned with the traversal path, reducing the pointer update to a simple increment by 1.

Therefore, the most optimized design choice is a hybrid approach: storing Matrix A in row-major order, Matrix B in column-major order, and Matrix C (the result) in row-major order. This guarantees sequential memory access for all three matrices.